

WebRTC Live Proctoring System Overview

The HRTA live proctoring system uses WebRTC (RTCPeerConnection) for one-way audio/video streaming from the candidate's browser to the Superadmin Command Center. Supabase Broadcast Channels act as the real-time signaling plane to negotiate SDP Offers, SDP Answers, and ICE Candidates.

Key Signaling Events:

1. Admin opens student card -> sends 'ADMIN_CONNECTED' signal.
2. Student receives signal -> initializes webcam capture, adds tracks, sends 'SDP_OFFER'.
3. Admin receives offer -> sets remote description, generates 'SDP_ANSWER'.
4. Candidate receives answer -> establishes connection and starts streaming.

Section A: Student WebRTC and Camera Controls (ExamInterface.jsx)

--- [File: ExamInterface.jsx | Lines 40 - 55] ---

```
40 | // Proctoring Refs
41 | const localStreamRef = React.useRef(null);
42 | const peerConnectionRef = React.useRef(null);
43 | const proctorChannelRef = React.useRef(null);
44 | const studentIceQueueRef = React.useRef([]);
45 | const maxWarningsRef = React.useRef(5);
46 | const isConnectingRef = React.useRef(false);
47 | const lockChannelRef = React.useRef(null);
48 | const isSubmittedRef = React.useRef(false);
49 | const isSubmittingRef = React.useRef(false);
50 | const [cameraAccessLost, setCameraAccessLost] = useState(false);
51 | const [cameraRetryLoading, setCameraRetryLoading] = useState(false);
52 | const cameraAccessLostRef = React.useRef(false);
53 | const tabSwitchCountRef = React.useRef(0);
54 |
55 | // Proctoring Violation Lock States
```

--- [File: ExamInterface.jsx | Lines 498 - 520] ---

```
498 | const canvasCtx = offscreenCanvas.getContext('2d');
499 |
500 | const startCameraAndProctoring = async () => {
501 |   try {
502 |     // Capture video and audio stream silently with fallback to video-only if mic
    rophone is missing/denied
503 |     try {
504 |       stream = await navigator.mediaDevices.getUserMedia({
505 |         video: { width: 640, height: 480, frameRate: 15 },
506 |         audio: { echoCancellation: true, noiseSuppression: true }
507 |       });
508 |     } catch (audioErr) {
509 |       console.warn("Failing back to video-only capture:", audioErr);
510 |       stream = await navigator.mediaDevices.getUserMedia({
511 |         video: { width: 640, height: 480, frameRate: 15 }
512 |       });
513 |     }
514 |     localStreamRef.current = stream;
515 |
516 |     // Bind stream to offscreen video element for frame analysis
517 |     offscreenVideo.srcObject = stream;
518 |     offscreenVideo.play().catch(e => console.warn("Offscreen video play failed:",
    e));
519 |
520 |     // Track ended and mute listeners (Self-healing and Logging)
```

HRTA PORTAL - WEBRTC LIVE PROCTORING SYSTEM SOURCE CODE EXPORT

Contains: ExamInterface.jsx (Student-side) & AdminDashboard.jsx (Admin-side)

--- [File: ExamInterface.jsx | Lines 670 - 770] ---

```
670 |         })
671 |         }).catch(err => console.warn("Failed to audit CHANNEL_AUTH_FAILURE:", e
    rr));
672 |         return;
673 |     }
674 |
675 |     if (type === "ADMIN_CONNECTED") {
676 |         if (isConnectingRef.current) {
677 |             console.log("Ignored ADMIN_CONNECTED: WebRTC connection setup already
in progress.");
678 |             return;
679 |         }
680 |         isConnectingRef.current = true;
681 |
682 |         if (peerConnectionRef.current) {
683 |             peerConnectionRef.current.close();
684 |             peerConnectionRef.current = null;
685 |         }
686 |         studentIceQueueRef.current = [];
687 |
688 |         // Wait up to 10 seconds for the camera stream to be active/loaded
689 |         const activeStream = await getActiveStream();
690 |         if (!activeStream) {
691 |             console.warn("WebRTC aborted: No active camera stream available.");
692 |             isConnectingRef.current = false;
693 |             return;
694 |         }
695 |
696 |         const pc = new RTCPeerConnection({
697 |             iceServers: [
698 |                 { urls: "stun:stun.l.google.com:19302" },
699 |                 { urls: "stun:stun1.l.google.com:19302" },
700 |                 { urls: "stun:stun2.l.google.com:19302" },
701 |                 { urls: "stun:stun3.l.google.com:19302" },
702 |                 { urls: "stun:stun4.l.google.com:19302" }
703 |             ]
704 |         });
705 |         peerConnectionRef.current = pc;
706 |
707 |         pc.onconnectionstatechange = () => {
708 |             if (pc.connectionState === "connected") {
709 |                 fetch(`${apiBaseUrl}/api/audit-log`, {
710 |                     method: 'POST',
711 |                     headers: getHeaders(),
712 |                     body: JSON.stringify({
713 |                         userId: userId || 'Unknown',
714 |                         userRole: 'student',
715 |                         displayName: studentRef.current?.full_name || 'Student',
716 |                         action: 'SIGNALING_ESTABLISHED',
717 |                         details: { exam_id: examId, connection_state: pc.connectionStat
e }
718 |                     })
719 |                 }).catch(err => console.warn("Failed to audit SIGNALING_ESTABLISHED
:", err));
720 |             } else if (pc.connectionState === "failed" || pc.connectionState ===
"disconnected") {
721 |                 if (!isSubmittedRef.current) {
722 |                     fetch(`${apiBaseUrl}/api/audit-log`, {
723 |                         method: 'POST',
724 |                         headers: getHeaders(),
725 |                         body: JSON.stringify({
```

HRTA PORTAL - WEBRTC LIVE PROCTORING SYSTEM SOURCE CODE EXPORT

Contains: ExamInterface.jsx (Student-side) & AdminDashboard.jsx (Admin-side)

```
726 |         userId: userId || 'Unknown',
727 |         userRole: 'student',
728 |         displayName: studentRef.current?.full_name || 'Student',
729 |         action: 'STREAM_DISCONNECTED',
730 |         details: { exam_id: examId, connection_state: pc.connectionSt
    ate }
731 |     })
732 |     }).catch(err => console.warn("Failed to audit STREAM_DISCONNECTED
    :", err));
733 |     }
734 |   }
735 | };
736 |
737 | // Add video and audio tracks (Strictly One-Way: Candidate sends to adm
    in)
738 | activeStream.getTracks().forEach((track) => {
739 |   pc.addTrack(track, activeStream);
740 | });
741 |
742 | pc.onicecandidate = (event) => {
743 |   if (event.candidate && proctorChannelRef.current) {
744 |     proctorChannelRef.current.send({
745 |       type: "broadcast",
746 |       event: "signal",
747 |       payload: { type: "ICE_CANDIDATE", sender: "student", data: event.
    candidate }
748 |     });
749 |   }
750 | };
751 |
752 | const offer = await pc.createOffer();
753 | await pc.setLocalDescription(offer);
754 | channel.send({
755 |   type: "broadcast",
756 |   event: "signal",
757 |   payload: { type: "SDP_OFFER", sender: "student", data: offer }
758 | });
759 | isConnectingRef.current = false;
760 |
761 | } else if (type === "SDP_ANSWER" && peerConnectionRef.current) {
762 |   try {
763 |     await peerConnectionRef.current.setRemoteDescription(new RTCSessionDe
    scription(data));
764 |     while (studentIceQueueRef.current.length > 0) {
765 |       const candidate = studentIceQueueRef.current.shift();
766 |       if (candidate) {
767 |         await peerConnectionRef.current.addIceCandidate(new RTCIceCandida
    te(candidate)).catch(e =>
768 |           console.warn("Error processing queued candidate on student:", e
    )
769 |         );
770 |       }
    }
```

Section B: Admin WebRTC and Feed Monitor (AdminDashboard.jsx)

--- [File: AdminDashboard.jsx | Lines 30 - 40] ---

```
30 |   const [ticketsError, setTicketsError] = useState(false);
31 |
32 |   // Proctoring States
33 |   const [activeSessions, setActiveSessions] = useState([]);
34 |   const [monitoringStudent, setMonitoringStudent] = useState(null);
```

HRTA PORTAL - WEBRTC LIVE PROCTORING SYSTEM SOURCE CODE EXPORT

Contains: ExamInterface.jsx (Student-side) & AdminDashboard.jsx (Admin-side)

```
35 |   const [remoteStream, setRemoteStream] = useState(null);
36 |   const [monitorStatus, setMonitorStatus] = useState("Connecting...");
37 |   const [timeTick, setTimeTick] = useState(0);
38 |
39 |   // Proctoring Unlock & Audio Mute States
40 |   const [isStudentLocked, setIsStudentLocked] = useState(false);
--- [File: AdminDashboard.jsx | Lines 145 - 250] ---
145 |       action: 'ADMIN_MONITOR_START',
146 |       details: {
147 |         monitored_student_id: session.student_id,
148 |         monitored_student_name: session.students?.full_name,
149 |         exam_id: session.exam_id,
150 |         exam_title: session.exams?.title
151 |       }
152 |     })
153 |   }).catch(err => console.warn("Failed to audit ADMIN_MONITOR_START:", err));
154 | } catch (logErr) {
155 |   console.warn("Failed to audit ADMIN_MONITOR_START:", logErr);
156 | }
157 |
158 | const studentId = session.student_id;
159 | const channelName = `exam_proctor_${studentId}`;
160 | const channel = supabase.channel(channelName);
161 | proctorChannelRef.current = channel;
162 |
163 | const pc = new RTCPeerConnection({
164 |   iceServers: [
165 |     { urls: "stun:stun.l.google.com:19302" },
166 |     { urls: "stun:stun1.l.google.com:19302" },
167 |     { urls: "stun:stun2.l.google.com:19302" },
168 |     { urls: "stun:stun3.l.google.com:19302" },
169 |     { urls: "stun:stun4.l.google.com:19302" }
170 |   ]
171 | });
172 | peerConnectionRef.current = pc;
173 |
174 | const iceCandidateQueue = [];
175 |
176 | pc.ontrack = (event) => {
177 |   if (event.streams && event.streams[0]) {
178 |     setRemoteStream(event.streams[0]);
179 |     setMonitorStatus("Live");
180 |   }
181 | };
182 |
183 | pc.onicecandidate = (event) => {
184 |   if (event.candidate && proctorChannelRef.current) {
185 |     proctorChannelRef.current.send({
186 |       type: "broadcast",
187 |       event: "signal",
188 |       payload: { type: "ICE_CANDIDATE", sender: "admin", data: event.candidate }
189 |     });
190 |   }
191 | };
192 |
193 | pc.oniceconnectionstatechange = () => {
194 |   if (pc.iceConnectionState === "disconnected" || pc.iceConnectionState === "failed") {
195 |     setMonitorStatus("Stream Interrupted");
196 |   } else if (pc.iceConnectionState === "closed") {
197 |     setMonitorStatus("Disconnected");
```

HRTA PORTAL - WEBRTC LIVE PROCTORING SYSTEM SOURCE CODE EXPORT

Contains: ExamInterface.jsx (Student-side) & AdminDashboard.jsx (Admin-side)

```
198 |     }
199 |   };
200 |
201 |   channel
202 |     .on("broadcast", { event: "signal" }, async ({ payload }) => {
203 |       const { type, sender, data } = payload;
204 |       if (sender === "student") {
205 |         if (type === "SDP_OFFER") {
206 |           try {
207 |             setMonitorStatus("Connecting...");
208 |             await pc.setRemoteDescription(new RTCSessionDescription(data));
209 |
210 |             const answer = await pc.createAnswer();
211 |             await pc.setLocalDescription(answer);
212 |
213 |             channel.send({
214 |               type: "broadcast",
215 |               event: "signal",
216 |               payload: { type: "SDP_ANSWER", sender: "admin", data: answer }
217 |             });
218 |
219 |             // Process any queued ICE candidates
220 |             while (iceCandidateQueue.length > 0) {
221 |               const candidate = iceCandidateQueue.shift();
222 |               if (candidate) {
223 |                 await pc.addIceCandidate(new RTCIceCandidate(candidate)).catch(e =>
224 |
225 |                   console.warn("Error processing queued candidate:", e)
226 |                 );
227 |               }
228 |             } catch (err) {
229 |               console.error("Error establishing connection with offer:", err);
230 |               setMonitorStatus("Connection Failed");
231 |             }
232 |           } else if (type === "ICE_CANDIDATE") {
233 |             try {
234 |               if (data) {
235 |                 if (pc.remoteDescription) {
236 |                   await pc.addIceCandidate(new RTCIceCandidate(data));
237 |                 } else {
238 |                   iceCandidateQueue.push(data);
239 |                 }
240 |               }
241 |             } catch (err) {
242 |               console.error("Error setting ICE candidate:", err);
243 |             }
244 |           } else if (type === "STUDENT_CAMERA_RECOVERED") {
245 |             console.log("Student camera recovered. Re-initializing proctor feed.");
246 |             startMonitoring(session);
247 |           } else if (type === "PROCTORING_VIOLATION") {
248 |             setIsStudentLocked(true);
249 |             setLockedReason(data?.reason || "Exam interface locked due to proctoring
violation.");
250 |             toast.error(`Violation Detected: ${data?.reason || "Exam Locked"}`);
```

--- [File: AdminDashboard.jsx | Lines 1996 - 2038] ---

```
1996 |         </span>
1997 |         Active
1998 |       </span>
1999 |     ) : (
2000 |       <span className="inline-flex items-center gap-1.5 bg-sl
```

HRTA PORTAL - WEBRTC LIVE PROCTORING SYSTEM SOURCE CODE EXPORT

Contains: ExamInterface.jsx (Student-side) & AdminDashboard.jsx (Admin-side)

```

    ate-500/10 border border-white/5 text-slate-500 px-2.5 py-0.5 rounded-full text-[10px
    ] font-bold">
2001 |         <span className="h-1.5 w-1.5 rounded-full bg-slate-60
0"></span>
2002 |             Offline
2003 |         </span>
2004 |     )}
2005 | </td>
2006 | </tr>
2007 | );
2008 | }}}
2009 | </tbody>
2010 | </table>
2011 | })
2012 | </div>
2013 | </div>
2014 | })
2015 |
2016 | {/* Live Proctoring Violations & Audit Logs Feed */}
2017 | {auditError ? (
2018 |     <div className="bg-amber-500/5 border border-amber-500/15 rounded-2xl p-6 s
hadow-xl text-xs font-semibold text-amber-300 mt-8">
2019 |         <div className="flex items-start gap-3.5">
2020 |             <span className="text-xl">??</span>
2021 |             <div>
2022 |                 <h3 className="font-bold text-amber-400 text-sm mb-1 uppercase tracki
ng-wider">Live Proctoring & Audit Feed: Table Error</h3>
2023 |                 <p className="text-slate-400 leading-relaxed">
2024 |                     Unable to fetch from the audit logs table. Please verify that the <
code>audit_logs</code> table exists and contains correct columns.
2025 |                 </p>
2026 |             </div>
2027 |         </div>
2028 |     </div>
2029 | ) : (
2030 |     <div className="bg-transparent border border-white/5 rounded-2xl shadow-2xl
overflow-hidden mt-8">
2031 |         <div className="bg-transparent border-b border-white/5 px-6 py-4 flex jus
tify-between items-center">
2032 |             <div>
2033 |                 <h2 className="font-bold uppercase tracking-wider text-xs text-white
flex items-center gap-2">
2034 |                     <span className="relative flex h-2 w-2">
2035 |                         <span className="animate-ping absolute inline-flex h-full w-full
rounded-full bg-red-405 opacity-75"></span>
2036 |                         <span className="relative inline-flex rounded-full h-2 w-2 bg-red
-500"></span>
2037 |                     </span>
2038 |                     Live Candidate Proctoring & Security Audit Logs
```